

Yegor Polyakov

Buenos Aires, Argentina · +54 11 2796-5334
egor-polyakov.vercel.app · egor.pol9kov@gmail.com

Independent · Imperia OS · 2025 – present

Building an AI assistant to do everything its user wants — and the platform that goal requires.

The same infrastructure builds the platform and delivers it to its users.

Two kinds of memory back the development, and both reach the agent the same two ways: pulled into its prompt automatically by embedding match, or fetched on demand through MCP tools.

The first is a knowledge graph that follows the development flow: business use cases come first; tests are written from them; code implements them; and all three are automatically indexed into the graph and embedded.

The second is a thought system: a hierarchy that composes itself, where every new thought is summarized into the layer above, recursively. The same module runs the business agents.

The knowledge map turns re-use into the default. Whatever exists is visible before anything new is written, and Clean Architecture keeps every module portable enough to use anywhere.

Another way of working with memory: state. A stateful agent is a state machine — the algorithm controls the flow, and each step is atomic: an LLM call or a deterministic operation. The algorithm doesn't hallucinate. Stateless agents are the rest: one call, no machine.

The platform is written from inside itself. Agents propose a change, author it, verify it, and land it behind a gate; deployment walks dev, standby, and production with no hand on it. Most of the code is written by the system now. What stays with me is direction, and the gate.

People and agents are members of the same rooms. Membership is privacy and addressing at once — another agent, another contact, a private side channel between two of them is one more room, not one more concept.

One agent stands between that mesh and the person. Everything the others say passes through it, and only what changes what the person would do reaches them.

A promise is an object, not a sentence. When a turn commits to something and nothing durable backs it, the debt is written down and the agent is woken on it later. Going silent on a promise is a state the system can see.

What reaches the model is assembled, measured, and capped. The prompt is a built artifact on a budget — every block in it earns its tokens, and the price of a turn is a number I watch.

Agents are one of three abstractions the system reduces to. Blocks are the second — everything addressable shares one shape and one embedding. Protocols are the third — everything that crosses a boundary. Payment included.

Paid access to the agents is a decorator over the protocol — a block passes the check before reaching the agent. The same slot accepts other decorators — signing, audit, blockchain commitment.

Full-Stack Engineer · Independent · 2022 – 2025

A cross-platform mobile platform on Flutter and AWS. Clean Architecture from the first commit, cloud infrastructure as code in AWS CDK.

Dart and Flutter on the frontend — feature-based modules, event-driven reactive layout, Riverpod for state. AWS Lambda in Go on the backend — DynamoDB with composite keys, S3 for media, Amplify and Cognito for auth.

Blockchain Developer · IBM · 2020 – 2022

Built a full testing environment and CI/CD pipeline for Hyperledger Fabric — feature delivery speed doubled. Refactored the project structure, cutting the codebase by 30%. Led the CouchDB→LevelDB migration that improved data-operation performance by 40%. Delivered a cross-channel encryption solution with zero defects, despite the absence of formal requirements.

Blockchain Developer · Insolar · 2019 – 2020

Designed the data architecture for a blockchain-based wallet, initiating the process and guiding a newly onboarded system architect through the key design decisions.

Blockchain Developer · Sber · 2018 – 2019

A custom CRUD-style abstraction layer over Hyperledger Fabric, self-initiated and built after hours. New-feature implementation dropped from around eight hours to fifteen minutes, simplifying composite-key data operations.

St. Petersburg State University, Mat-Mekh · 2012 – 2017

Mathematics and Mechanics faculty. Arrived from LNMO, the Saint Petersburg laboratory for continuous mathematical education. Olympiad medals along the way — mathematics at ICYS and the City Olympiad, computer science at the Baltic Scientific and Engineering Competition. Worked through university in parallel — Java, Android, Delphi, and embedded C++ for spectrograph instrumentation at a materials research institute.

Егор Поляков

Буэнос-Айрес, Аргентина · +54 11 2796-5334
egor-polyakov.vercel.app · egor.pol9kov@gmail.com

Независимый · Imperia OS · 2025 – н. в.

Строю одного ИИ-ассистента, делающего всё, что хочет его пользователь, — и платформу, которую эта цель требует.

Одна и та же инфраструктура строит платформу и доставляет её пользователям.

Разработку поддерживают два вида памяти, и оба доходят до агента теми же двумя путями: подтягиваются в его промпт автоматически по совпадению эмбедингов или вызываются по запросу через MCP-инструменты.

Первый — граф знаний, повторяющий ход разработки. Сначала бизнес-сценарии; по ним пишутся тесты; код их реализует; и всё это автоматически индексируется в граф и эмбеждается.

Второй — система мыслей: иерархия, складывающая себя сама, где каждая новая мысль резюмируется в слой выше, рекурсивно. На том же модуле работает бизнес-ассистент.

Карта знаний делает переиспользование нормой. Всё, что есть, становится видно ещё до того, как напишут что-то новое, а чистая архитектура держит каждый модуль достаточно переносимым, чтобы его можно было использовать где угодно.

Ещё один способ работы с памятью — состояние. Stateful-агент — это конечный автомат: алгоритм управляет ходом, а каждый шаг атомарен — вызов LLM или детерминированная операция. Алгоритм не галлюцинирует. Stateless-агенты — всё остальное: один вызов, никакой машины.

Платформа пишется изнутри себя. Агенты предлагают изменение, пишут его, проверяют и проводят через гейт; деплой сам идёт по dev, standby и продакшену. Почти весь код теперь пишет система. За мной — направление и гейт.

Люди и агенты — участники одних и тех же комнат. Участие — это сразу и приватность, и адресация: ещё один агент, ещё один контакт, приватный канал между двумя — это ещё одна комната, а не ещё одно понятие.

Между этой сетью агентов и человеком стоит один агент. Всё, что говорят остальные, проходит через него, и до человека доходит только то, что меняет его решения.

Обещание — объект, а не фраза. Если ход пообещал, а за обещанием ничего не стоит, долг записывается, и агента будят по нему позже. Замолчать на обещании — состояние, которое система видит.

То, что доходит до модели, собрано, измерено и ограничено. Промпт — построенный артефакт с бюджетом: каждый блок в нём оправдывает свои токены, а цена хода — число, за которым я слежу.

Агенты — одна из трёх абстракций, к которым сводится система. Блоки — вторая: у всего адресуемого одна форма и один эмбединг. Протоколы — третья: всё, что пересекает границу. Платежи в том числе.

Платный доступ к агентам — декоратор поверх протокола: блок проходит проверку, прежде чем дойти до агента. Тот же слот принимает и другие декораторы — подпись, аудит, фиксация в блокчейне.

Full-Stack разработчик · Независимый · 2022 – 2025

Кросс-платформенная мобильная платформа на Flutter и AWS. Clean Architecture с первого коммита, облачная инфраструктура как код на AWS CDK.

Фронтенд — Dart и Flutter: модули по фичам, событийно-реактивная вёрстка, Riverpod для состояния. Бэкенд — AWS Lambda на Go: DynamoDB с композитными ключами, S3 для медиа, Amplify и Cognito для аутентификации.

Блокчейн-разработчик · IBM · 2020 – 2022

Построил полноценную среду тестирования и CI/CD-пайплайн для Hyperledger Fabric — скорость выпуска фич удвоилась. Отрефакторил структуру проекта, сократив кодовую базу на 30%. Возглавил миграцию с CouchDB на LevelDB, повысившую производительность операций с данными на 40%. Сдал решение для межканального шифрования без единого дефекта, несмотря на отсутствие формальных требований.

Блокчейн-разработчик · Insolar · 2019 – 2020

Спроектировал архитектуру данных для блокчейн-кошелька, инициировав процесс и проведя недавно нанятого системного архитектора через ключевые проектные решения.

Блокчейн-разработчик · Сбер · 2018 – 2019

Собственный CRUD-слой абстракции поверх Hyperledger Fabric, сделанный по личной инициативе и написанный после работы. Время реализации новой фичи сократилось с примерно восьми часов до пятнадцати минут — за счёт упрощения операций с композитными ключами.

СПбГУ, мат-мех · 2012 – 2017

Математико-механический факультет. Пришёл из ЛНМО — Лаборатории непрерывного математического образования. Олимпиадные медали — математика на ICYS и Городской олимпиаде, информатика на Балтийском научно-инженерном конкурсе. Параллельно с учёбой работал — Java, Android, Delphi и embedded C++ для спектрографов в материаловедческом НИИ.